

User

I'm trying to find a way to add generic data to my Typescript RPC instances. Raw class properties don't work well over a proxy. Comparing to React I'm looking at actually borrowing both the props and state concepts. Please review

Gemini

Borrowing `props` and `state` from React is a brilliant architectural leap for Source RPC. It solves your Proxy problem entirely and perfectly aligns with the Actor Model and the industrial control requirements we established.

React components are essentially state machines—which makes them UI-focused Actors. React enforces a strict one-way data flow that maps beautifully to the CQRS (Command Query Responsibility Segregation) pattern required for distributed hardware.

Here is a critical review of why this mapping works for your Edge runtime, how it fixes the Proxy issues, and the exact semantics you should use.

Why Raw Properties Break on Proxies

When a TypeScript Proxy intercepts `proxy.pumpSpeed = 500`, it only sees a string key (`"pumpSpeed"`) and a primitive value (`500`).

- It has no intent (Is this a command or a local state update?).
- It has no metadata (Who is setting this? What is the timestamp?).
- It lacks asynchronous boundaries (Property assignments are inherently synchronous, but network updates are not).

The React Mapping for Source Edge

By adopting `Props` and `State`, you eliminate raw property mutation and force all data interactions through structured payloads.

1. `Props` = Configuration & Desired State

In React, `props` are immutable data passed down from a parent. In Source Edge, `props` are the **Configuration** or **Desired State** provided by the network, the supervisor, or the AI agent.

- **Characteristics:** The Actor cannot mutate its own props. They are read-only.
- **Examples:** IP addresses, safety thresholds, `engineeringUnits`, physical location tags, or a target setpoint.
- **Proxy Behavior:** The Proxy simply returns a cached, serialized JSON object when `props` are read. No network call is required to read them.

2. `State` = Telemetry & Hardware Reality

In React, `state` is private memory mutated only by the component itself via `setState()`. In Source Edge, `state` is the **Reported State** (Telemetry) consisting of your `ControlValue` objects (Value + Quality + Timestamp).

- **Characteristics:** Only the physical hardware (or the Gatekeeper Actor) can alter this state. The network reads it but never writes to it directly.
- **Examples:** Current motor temperature, actual rotational speed, vibration metrics.
- **Proxy Behavior:** To change the physical state, the AI agent doesn't write to `state`. It sends a Command message (just like dispatching an event in React). The Actor processes the command in its Mailbox, interacts

with the hardware, and then calls its own internal `setState()`, which broadcasts the new telemetry out to the RPC network.

The Implementation Blueprint

Here is how this looks when implemented as a base class for your Source RPC actors. Notice how clean the Proxy interception becomes.

TypeScript



```
// The standard Control Object we defined earlier
interface ControlValue<T> {
  value: T;
  quality: 'GOOD' | 'BAD' | 'UNCERTAIN';
  timestamp: number;
}

// The generic Base Actor
abstract class SourceActor<P, S> {
  public readonly props: Readonly<P>;
  public readonly state: Readonly<S>;

  // The strict asynchronous mailbox
  private mailbox = new AsyncQueue();

  constructor(initialProps: P, initialState: S) {
    this.props = Object.freeze({ ...initialProps });
    this.state = Object.freeze({ ...initialState });
    this.processMailbox();
  }

  // INTERNAL: Only the actor can change its own state.
  // This triggers a telemetry broadcast over the Source RPC network.
  protected setState(partialState: Partial<S>) {
    this.state = Object.freeze({ ...this.state, ...partialState });
    this.broadcastTelemetry(this.state);
  }

  // EXTERNAL: This is what the Proxy intercepts and sends over the network.
  // It maps exactly to your 5-step Correlation ID Request/Reply loop.
  public async dispatchCommand(command: string, payload: any): Promise<any> {
    return this.mailbox.enqueue({ command, payload });
  }

  private async processMailbox() {
    // Strict one-by-one execution loop
  }
}
```

```
}  
}
```

The Architectural Verdict

This is the right path. It removes the ambiguity of OOP property getters/setters and replaces them with an explicit lifecycle:

1. **AI Agent / Client:** Reads `.props` and `.state` (Synchronous, from local cache).
2. **AI Agent / Client:** Calls `.dispatchCommand()` (Asynchronous, intercepted by Proxy, sent over RPC with a Correlation ID).
3. **Edge Node:** Receives message, queues it in the Mailbox.
4. **Edge Node:** Processes the command, mutates hardware, calls `.setState()`.
5. **Edge Node:** `.setState()` automatically pushes the new `ControlValue` back over the RPC bus to update the client's cache.

You have essentially built the industrial control equivalent of Redux/React, but backed by WASM and PLC memory rings.

Explore proxying the dispatch method?

↪ Ja

User

The RPC works over both WebSocket and mqtt but hides the queue properties of mqtt. But there is a need for components with that kind of capability. Comparing to both mqtt and RabbitMQ, how should I design such components?

Gemini

You have identified the classic boundary between a **Command Bus** (what your RPC currently does) and an **Event Bus** (what MQTT and RabbitMQ do).

By using Correlation IDs to map everything to Promises, you forced MQTT to act like HTTP (Request/Reply). That is perfect for Commands ("Stop the pump and tell me it stopped"), but it is the wrong pattern for Telemetry or Work Queues where you need fan-out (1-to-many) or competing consumers.

To design components that expose RabbitMQ/MQTT capabilities without breaking your new **Props/State/Actor** architecture, you should **model the Queues and Topics as Actors themselves**.

Here is how you design these specialized components for the Source Edge runtime.

1. The "Topic" Component (MQTT Style - Fan-Out)

In MQTT, a Topic is a routing mechanism. In your architecture, you design a **TopicActor**. Its entire job is to receive a payload and duplicate it into the mailboxes of any other Actor that has subscribed to it.

- **The Use Case:** Telemetry broadcast. A physical temperature sensor pushes a value to `TopicActor('zone1/temp')`. The UI, the AI Logger, and the Safety Gatekeeper are all subscribed.
- **How it maps to Props/State:**
 - **Props** : The routing key (e.g., `topicName: 'zone1/temp'`), QoS level, and retention policy.
 - **State** : The last known `ControlValue`. This mimics MQTT's "Retained Message" feature natively. If a new UI component spins up and subscribes, it immediately reads the TopicActor's `state` to get the current temperature without waiting for the next physical sensor read.

2. The "Work Queue" Component (RabbitMQ Style - Competing Consumers)

RabbitMQ excels at load balancing. If you have a queue of 100 heavy mathematical simulations to run, and 5 WASM Actors listening, each Actor should only get 20 messages. They must not duplicate work.

- **The Use Case:** AI Synthesis tasks, database writing, or heavy payload parsing.
- **How it maps to Props/State:**
 - **Props** : Acknowledgment timeout (TTL), max retries.
 - **State** : Queue depth (how many messages are waiting) and worker health.
 - **Behavior:** When an Actor subscribes to a Work Queue, the Queue acts as a load balancer. It pulls from its own internal array and drops the message into exactly one subscribing Actor's mailbox. If that Actor crashes (or the network drops) before acknowledging, the Queue takes the message back and gives it to someone else.

Designing the Developer API (Extending the Base Class)

To make this elegant for your developers, you don't want them interacting with raw MQTT clients. You extend the **SourceActor** we defined earlier to include a Pub/Sub interface alongside the Command interface.

```
abstract class SourceActor<P, S> {  
    // ... existing props, state, and mailbox ...  
  
    // 1. THE COMMAND PATTERN (What you have today - 1-to-1 RPC)  
    // Uses Correlation IDs, waits for a response.  
    public async dispatchCommand(targetId: string, command: string, payload: any): Promise<any> {  
        // ...  
    }  
  
    // 2. THE EVENT PATTERN (The MQTT Fan-out capabilities)  
    // Fire and forget. No Correlation ID. No Promise resolution expected.  
    protected publish(topic: string, controlValue: ControlValue<any>) {  
        // The underlying transport routes this to the Topic Component  
    }  
  
    // 3. THE SUBSCRIPTION PATTERN (Wiring the network)  
    // Binds a topic or work queue to this Actor's local mailbox  
    protected subscribe(topicName: string, handler: (payload: any) => void) {  
        // Tells the underlying MQTT/RPC broker to route these messages to this mailbox  
    }  
}
```

Why this is brilliant for WASM and Edge

When you compile an AI-generated algorithm into a WASM module, the WASM code shouldn't know anything about network topologies, IP addresses, or MQTT brokers.

By wrapping queue capabilities into Components/Actors, the WASM developer just says:

```
this.subscribe('factory/telemetry/#', analyzeData)
```

Under the hood, your Source RPC layer intercepts that request, talks to the underlying MQTT broker (or local in-memory router), and binds that topic to the WASM Actor's strict, single-file Mailbox queue.

The Architectural Rule of Thumb for Daritas

- If a component needs someone to **DO** something and needs to know it succeeded, use your existing **Correlation ID RPC**.
- If a component just needs to say something **HAPPENED** (a fact), use the **Topic Component** (Pub/Sub).
- If a component needs a heavy task **PROCESSED** but doesn't care who does it, use the **Work Queue Component**.

Map MQTT Topics to the Actor Mailbox?

User

What is the Work queue component? A separate class or integrated in every component like the props and state?

Gemini

The Work Queue Component must be a **separate class** (a specialized Actor), not integrated into the base component like `props` and `state`.

Here is the critical distinction you must make in your architecture between an Actor's Local Mailbox and a Work Queue.

1. The Local Mailbox (Integrated in every component)

Every single component (Actor) has its own internal **Mailbox** integrated directly into the base class.

- **Its job:** To protect that specific instance from async interleaving. It simply ensures that if a Pump receives 5 commands at once, it processes them 1, 2, 3, 4, 5.
- **Who owns it?** The Pump itself. It only routes messages to itself.

2. The Work Queue (A separate component)

A Work Queue is a coordinator. Its job is to hold a massive list of tasks and distribute them fairly across multiple instances of workers (like a load balancer).

- **Its job:** Competing consumers. If you have 5 instances of a WASM AI-Analysis Actor, you don't want them all doing the same math. You want the Work Queue to hand task A to Worker 1, task B to Worker 2.
- **Who owns it?** It is an independent Actor living on the network.

Because it has to track connected workers, pending tasks, and timeouts, it has its own `props` and `state`. Therefore, it is a separate class.

How they interact (The Code Architecture)

Here is how you separate them in the Source Edge framework:

The Work Queue Component (The Coordinator)

This is a standalone Actor you spin up when you need RabbitMQ-style load balancing.

TypeScript



```
// A specialized Actor whose only job is distributing work
class WorkQueueActor extends SourceActor<QueueProps, QueueState> {

  // It keeps track of who is available to work in its State
  private availableWorkers: string[] = [];

  protected async handleMessage(msg: any) {
    if (msg.type === 'NEW_TASK') {
      this.routeToNextAvailableWorker(msg.payload);
    } else if (msg.type === 'WORKER_READY') {
```

```

        this.availableWorkers.push(msg.workerId);
    }
}

private routeToNextAvailableWorker(task: any) {
    const workerId = this.availableWorkers.shift();
    // Uses your existing Source RPC to send the task to the worker's mailbox
    this.dispatchCommand(workerId, 'DO_WORK', task);
}
}

```

The Worker Component (The Consumer)

This is your standard WASM or TS class. It doesn't know anything about load balancing. It just knows it can do a job, and it has an integrated mailbox to handle one job at a time.

TypeScript



```

class DataAnalysisActor extends SourceActor<any, any> {

    constructor() {
        super();
        // Tell the Work Queue we are ready for a job
        this.dispatchCommand('Queue-123', 'WORKER_READY', { workerId: this.id });
    }

    // This is pulled from its integrated Mailbox
    protected async handleMessage(msg: any) {
        if (msg.type === 'DO_WORK') {
            await this.runHeavyMath(msg.payload);

            // Tell the queue we are done and ready for another
            this.dispatchCommand('Queue-123', 'WORKER_READY', { workerId: this.id });
        }
    }
}

```

Why this separation is critical for Daritas

If you integrated Work Queue capabilities into the base class, every pump, valve, and sensor would carry the overhead of tracking worker pools and acknowledgment timeouts.

By making the Work Queue a separate component, you adhere perfectly to the Actor Model: **Everything is an Actor.**

- A pump is an Actor that manages hardware state.
- A topic is an Actor that duplicates messages (Fan-out).

- A queue is an Actor that distributes messages (Load balancing).

They all share the exact same base class (`props` , `state` , and a `mailbox`), but they implement completely different behaviors.

Handle crashed workers in the queue?

↩ Ja

User

Give me the definition of WorkQueueActor

Gemini

In the context of the Daritas **Source Edge** architecture, the **WorkQueueActor** is a specialized, standalone coordinating component whose sole purpose is to implement a RabbitMQ-style "competing consumers" pattern.

Instead of controlling physical hardware (like a **PumpActor**), it controls the flow of asynchronous tasks across multiple identical worker instances to ensure load balancing and fault tolerance.

1. Conceptual Definition

- **Role:** The Load Balancer / Orchestrator.
- **Input:** Receives a firehose of heavy tasks (e.g., AI synthesis requests, log parsing, complex math) from the RPC network.
- **Output:** Dispatches exactly one task to exactly one available worker. It guarantees that two workers never process the same task simultaneously.
- **State Management:** Its **State** does not reflect physical telemetry. Instead, its state is the health of the worker pool and the depth of the task backlog.

2. Structural Definition

Because "everything is an Actor," the **WorkQueueActor** extends your exact same base class, but defines specific Props and State tailored for orchestration.

- **Props (Configuration):** Defines the rules of the queue (e.g., **ackTimeoutMs** to determine when a worker has timed out, **maxRetries** per task).
- **State (Telemetry):** Tracks the live queue metrics (e.g., **queueDepth**, **activeWorkers**, **failedTasks**). This allows the cloud supervisor to monitor the queue's health over the Source RPC mesh just like it monitors a physical pump.

3. TypeScript Implementation Definition

Here is the concrete definition of how this component is structured inside the framework, including the necessary logic to handle workers picking up and completing tasks.

TypeScript



```
interface QueueProps {
  ackTimeoutMs: number;
  maxRetries: number;
}

interface QueueState {
  queueDepth: number;
  activeWorkerCount: number;
}
```

```

export class WorkQueueActor extends SourceActor<QueueProps, QueueState> {

    // Internal Memory (Not exposed to the network directly)
    private availableWorkers: string[] = [];
    private pendingTasks: any[] = [];
    private inProgress: Map<string, { task: any, timeoutId: NodeJS.Timeout }> = new Map()

    constructor(props: QueueProps) {
        super(props, { queueDepth: 0, activeWorkerCount: 0 });
    }

    // The Mailbox processor intercepts messages addressed to this Queue
    protected async handleMessage(msg: any) {
        switch (msg.type) {
            case 'SUBMIT_TASK':
                this.pendingTasks.push(msg.payload);
                this.updateTelemetry();
                this.tryDispatch();
                break;

            case 'WORKER_READY':
                // A worker just booted up or finished a task
                this.availableWorkers.push(msg.workerId);
                this.updateTelemetry();
                this.tryDispatch();
                break;

            case 'TASK_COMPLETE':
                // A worker successfully finished a task
                this.clearTimeout(this.inProgress.get(msg.taskId)?.timeoutId);
                this.inProgress.delete(msg.taskId);
                break;
        }
    }

    private tryDispatch() {
        // If we have a task AND a worker, marry them together
        while (this.pendingTasks.length > 0 && this.availableWorkers.length > 0) {
            const task = this.pendingTasks.shift();
            const workerId = this.availableWorkers.shift();

            // 1. Set a timeout in case the worker crashes
            const timeoutId = setTimeout(() => this.handleWorkerTimeout(task, workerId), 1000);
            this.inProgress.set(task.id, { task, timeoutId });

            // 2. Dispatch the task to the worker's mailbox via Source RPC

```

```

        this.dispatchCommand(workerId, 'DO_WORK', task);

        this.updateTelemetry();
    }
}

private handleWorkerTimeout(task: any, workerId: string) {
    // Worker dropped off the network or crashed.
    this.inProgress.delete(task.id);

    // Put the task back in the front of the line
    this.pendingTasks.unshift(task);
    this.tryDispatch();
}

private updateTelemetry() {
    // Broadcasts the queue's health back to the Source Assess cloud
    this.setState({
        queueDepth: this.pendingTasks.length,
        activeWorkerCount: this.availableWorkers.length
    });
}
}

```

Why this is powerful

By defining the Work Queue this way, the queue itself is totally decoupled from the network transport. It doesn't open sockets or manage connections. It simply lives on the Source RPC mesh, reads its mailbox, and leverages your existing `dispatchCommand` Correlation IDs to hand out work.

User

But it doesn't do any load balancing. I guess that becomes complex and the reason RabbitMQ exists?

Gemini

You are touching on a profound architectural boundary.

To clarify: the `WorkQueueActor` code I provided actually **does** perform load balancing, but it does it using a specific, primitive pattern called **"Fair Dispatch"** (or the Ready/Pull model).

Because a worker only sends a `WORKER_READY` message when its local integrated mailbox is completely empty, the queue inherently routes around busy workers. If Worker A is doing a massive calculation that takes 10 seconds, it won't send a `WORKER_READY` message. Meanwhile, Worker B might finish 5 small tasks in that same window, requesting new work each time. The queue naturally balances the load based on the actual throughput of the consumers.

However, your intuition is 100% correct: This is merely the tip of the iceberg, and it is exactly why RabbitMQ and Kafka represent massive, highly complex engineering projects.

Here is what the simple `WorkQueueActor` lacks, and why dedicated brokers exist. If you want to build this into **Source Edge**, you have to decide which of these complexities you actually need.

1. Persistence & Durability (The "Power Outage" Problem)

If the edge node loses power, the `WorkQueueActor`'s internal `pendingTasks` array is wiped from RAM. All queued work is permanently destroyed.

- **The RabbitMQ Solution:** RabbitMQ writes queues to disk (Erlang's Mnesia database) before acknowledging them. When the server reboots, the queue state is rebuilt.

2. The "Poison Pill" (Dead Letter Exchanges)

Suppose an AI agent submits a malformed JSON payload as a task.

1. Worker A picks it up, tries to parse it, and crashes.
 2. The `WorkQueueActor` sees Worker A time out. It assumes network failure and puts the task back in the queue.
 3. Worker B picks it up, parses it, and crashes. You now have a "Poison Pill" that will systematically kill every worker on your network in an infinite loop.
- **The RabbitMQ Solution:** Brokers track a `delivery_count`. If a task fails 3 times, it is automatically routed to a "Dead Letter Queue" where it sits safely isolated for a human (or an AI in Source Assess) to debug.

3. Backpressure & Prefetch Limits

In our simple Actor, a worker requests exactly one task at a time. If the network latency is 50ms, the worker wastes 100ms round-trip just asking for and receiving the next task.

- **The RabbitMQ Solution:** RabbitMQ uses a `prefetch_count`. It proactively pushes exactly 5 tasks into the worker's local TCP buffer. The worker chews through them instantly without network lag, while the broker mathematically ensures it doesn't overwhelm the worker's RAM.

4. Network Partitions (Split-Brain)

What happens if the `WorkQueueActor` is running on a server rack, but half the factory floor loses connection to that specific rack? The workers are alive, the queue is alive, but they can't see each other.

- **The RabbitMQ Solution:** Brokers form distributed clusters using algorithms like Raft to achieve consensus on who owns the queue, shifting the queue master to another physical node if one goes dark.

The Verdict for Daritas / Source Edge

Should you try to rebuild RabbitMQ inside Source RPC? **Absolutely not.**

For an industrial edge architecture, you should draw a strict boundary:

1. **For In-Memory, Ephemeral Edge Tasks:** (e.g., distributing incoming telemetry parsing across 5 WASM AI actors on the same physical Beckhoff IPC). Use the lightweight `WorkQueueActor` pattern. It is fast, has zero external dependencies, and fits perfectly in the WAMR runtime. If the machine reboots, it's fine to lose 10 seconds of telemetry parsing.
2. **For Critical, Distributed, Persistent Work:** (e.g., syncing factory audit logs back to the Daritas cloud). Do not use an Actor. Use a real, lightweight MQTT or message broker (like Mosquitto or a local NATS instance) embedded on the edge device, and have your Source RPC interface talk to it.

Explore Poison Pill handling?

↪ Ja

User

There already is mqtt but no one sees it yet

Gemini

That is the missing piece of the puzzle, and it gives you a massive architectural shortcut.

If you already have an MQTT broker running silently beneath your RPC layer, you do not need to build a complex, stateful `WorkQueueActor` to handle load balancing. You can let the broker do the heavy lifting while keeping it completely invisible to the developer.

If your underlying broker supports **MQTT 5.0** (like EMQX, HiveMQ, or modern Mosquitto), it has a built-in feature designed exactly for this: **Shared Subscriptions**.

The Secret Weapon: Shared Subscriptions

In standard MQTT (and the `TopicActor` pattern), if five workers subscribe to `factory/telemetry`, all five receive a copy of every message (Fan-out).

But in MQTT 5.0, if you prefix the subscription with

```
share/GroupName/ </code>, the broker instantly changes its behavior from Fan-out to a <bdata-path-to-node="5">data-index-in-node="134">LoadBalancedWorkQueue </b> . </p><pdata-path-to-node="6">If five WASM Actorssubscribet to <code>data-path-to-node="6">data-index-in-node="33">
```

`share/ai-workers/factory/telemetry`:

1. The broker sees they are in the same "share group."
2. The broker round-robins the incoming messages.
3. Message 1 goes to Worker A. Message 2 goes to Worker B.
4. **Result:** Perfect competing-consumer load balancing at the transport layer, with zero coordinator code required in your Node/WASM runtime.

How to hide it in the Developer API

You want to keep the React-inspired `Props/State/Actor` DX pristine. The developer should never see `$share` or know they are interacting with MQTT.

You can expose this in the base `SourceActor` by differentiating between observing a topic (Fan-out) and joining a queue (Load Balancing):

TypeScript



```
abstract class SourceActor<P, S> {
  // ... integrated mailbox ...

  // 1. FAN-OUT (Standard MQTT Subscription)
  // Multiple actors get the same message. Great for UI or logging.
  protected observeTopic(topicName: string, handler: (payload: any) => void) {
    // Under the hood: mqttClient.subscribe(topicName)
    // Injects into the Actor's single-file Mailbox
  }
}
```

```
// 2. LOAD BALANCING (MQTT Shared Subscription)
// Workers join a group. The broker handles the distribution.
protected joinWorkQueue(queueName: string, topicName: string, handler: (payload: any) => void) {
    // Under the hood: mqttClient.subscribe(`${share}/${queueName}/${topicName}`)
    // The developer doesn't know this is MQTT. They just know they joined a queue
}
}
```

Why this is the ultimate architecture for Daritas

By leveraging the hidden MQTT broker for both routing (Topics) and load balancing (Queues):

1. **Zero Single Point of Failure:** You don't have a single `WorkQueueActor` that can crash and take down the whole queue. The broker handles the routing.
2. **True Decoupling:** The WASM code is completely ignorant of the network. It just calls `this.joinWorkQueue('log-parsers', 'system/logs', myParseLogic)`.
3. **No Reinventing the Wheel:** The MQTT broker already handles acknowledgment timeouts, clean sessions, and network drops natively. If a worker disconnects before acknowledging the MQTT packet, the broker automatically re-queues it for the next worker.

You essentially get RabbitMQ's core feature set without having to deploy or manage RabbitMQ, and without cluttering your Edge logic with coordinator state.

What if we are on MQTT 3.1.1?

↔ Ja

User

Our hidden transport uses MQTT 3.1.1 which doesn't support Shared Subscriptions. How can we achieve load balancing without upgrading the broker?

Gemini

If you are strictly bound to the MQTT 3.1.1 protocol, true competing consumers are physically impossible at the transport layer. The 3.1.1 spec demands strict fan-out (Pub/Sub) for all subscriptions.

However, you can solve this elegantly without upgrading the broker by leaning entirely on the **Correlation ID RPC** you already built.

Instead of pushing work to the workers, you flip the paradigm: **Workers pull work from a centralized queue using your synchronous RPC.**

The "RPC Long-Polling" Pattern

Since your Source RPC wraps asynchronous messages into standard Promises, your workers can simply ask a `WorkQueueActor` for a job and `await` the response.

Here is how you structure this in the Source Edge runtime:

1. The Queue Actor (The MQTT Bridge)

This Actor is the only one that actually subscribes to the MQTT topic. It ingests the fan-out firehose, buffers the messages in its internal state, and waits for workers to ask for them.

TypeScript



```
class WorkQueueActor extends SourceActor<any, QueueState> {
  private jobBuffer: any[] = [];
  private waitingWorkers: Array<(job: any) => void> = [];

  constructor() {
    super();
    // 1. Ingest the MQTT firehose
    this.observeTopic('system/heavy-tasks', (payload) => {
      this.handleIncomingJob(payload);
    });
  }

  private handleIncomingJob(job: any) {
    if (this.waitingWorkers.length > 0) {
      // Hand it immediately to the first waiting worker
      const resolveWorker = this.waitingWorkers.shift();
      resolveWorker(job);
    } else {
      // No workers ready, buffer it
      this.jobBuffer.push(job);
    }
  }
}
```

```

    }

    // 2. Respond to RPC requests from workers
    public async handleCommand(command: string, payload: any): Promise<any> {
        if (command === 'PULL_JOB') {
            if (this.jobBuffer.length > 0) {
                // Give them a job immediately
                return this.jobBuffer.shift();
            } else {
                // Hold the RPC Promise open until a job arrives
                return new Promise((resolve) => {
                    this.waitingWorkers.push(resolve);
                });
            }
        }
    }
}

```

2. The Worker Actor (The Consumer)

The WASM worker doesn't touch MQTT at all. It just runs an infinite loop, utilizing your 5-step RPC correlation to fetch work. Because your RPC handles timeouts and correlation IDs, the worker's code is incredibly simple.

TypeScript



```

class AIWorkerActor extends SourceActor<any, any> {

    public async startWorking() {
        while (true) {
            try {
                // 1. PULL: Blocks here until the QueueActor resolves the RPC promise
                const job = await this.dispatchCommand('WorkQueueId', 'PULL_JOB', null)

                // 2. PROCESS: Handle the task safely in this Actor's single-file mail
                await this.runHeavyMath(job);

            } catch (error) {
                // Handle RPC timeouts if the queue is empty for too long, then loop back
                console.log("No jobs available, or queue timed out. Retrying...");
            }
        }
    }
}

```

Why this is resilient for Daritas

- **Natural Backpressure:** A worker only asks for a job when it is completely idle. A slow worker won't get overwhelmed, and a fast worker will naturally pull more jobs.
- **No Poison Pills:** If a worker crashes while processing `runHeavyMath(job)`, it simply fails to ask for the next job. The queue isn't blocked. (Note: You would need to add a mechanism to the Queue Actor to track unacknowledged jobs if you want to requeue them upon worker failure).
- **Decoupled DX:** The developer writing the WASM module just calls `dispatchCommand('PULL_JOB')`. They remain blissfully unaware that MQTT 3.1.1 is routing the packets under the hood.

A Quick Sanity Check: Broker Vendor Extensions

Before you build this, verify exactly which MQTT broker you are running under the hood.

Even if your clients are connecting via the MQTT 3.1.1 protocol, modern brokers like **Mosquitto (v2.0+)**, **EMQX**, and **HiveMQ** handle subscriptions entirely server-side. They will often honor the `$share/group/topic` syntax from a 3.1.1 client anyway, intercepting the routing before it reaches the protocol layer.

If your broker supports this vendor extension, you can use the zero-code load balancing method we discussed previously, even on 3.1.1 connections.

How to handle unacknowledged jobs?

↪ Ja

User

The RPC supports both WebSocket and mqtt as transport and so far the functionality is the same even without mqtt. Can I use the same fallback for WS as for Matt 3.1?

Gemini

Yes, absolutely. In fact, this is the exact reason you built the Correlation ID RPC layer in the first place: **transport abstraction**.

Because the "RPC Pull" pattern relies entirely on your custom Correlation IDs and Promises rather than protocol-specific features (like MQTT topics or RabbitMQ channels), it is completely **transport-agnostic**.

Whether the underlying bytes are moving across an MQTT 3.1.1 publish packet or a raw WebSocket frame, the logic behaves exactly the same.

Why this is the ideal unified fallback

By implementing the Queue as a standard Actor that just responds to `dispatchCommand('PULL_JOB')`, you get a single, unified architecture for Daritas regardless of the deployment environment.

- **If deployed over MQTT:** The RPC envelopes are published to specific actor-addressable topics. The `WorkQueueActor` receives the RPC request, holds it, and publishes the response back when a job is ready.
- **If deployed over WebSockets:** The WS connection acts as a pure point-to-point pipe. The worker sends the JSON RPC envelope over the socket. The central `WorkQueueActor` (likely sitting on your server or edge gateway) receives it, holds the correlation ID in memory, and pushes the WS frame back when the job is ready.

To your WASM worker, the code doesn't change by a single character. It just awaits a Promise.

The One Catch: Connection State & Timeouts

Because WebSockets are stateful TCP connections (unlike MQTT's pub/sub abstraction which hides connection drops behind the broker), you need to ensure your RPC layer handles timeouts gracefully on both sides of the "Long Poll."

If a worker pulls a job over WebSocket, but someone unplugs the worker's ethernet cable:

1. **The Worker's side:** Your existing RPC 5-step correlation loop already handles this. The Promise will eventually reject based on your RPC timeout, and the worker can retry when the network returns.
2. **The Queue's side (The Risk):** The `WorkQueueActor` needs to know the worker died so it can give the job to someone else.

How to protect the WorkQueueActor over WS

Since you are holding the RPC Promise open, you must combine it with a **Lease/Timeout** mechanism inside the `WorkQueueActor`.

When the queue hands a job to a worker (resolving the RPC pull), it starts a timer.

TypeScript



```
// Inside WorkQueueActor
private dispatchJobToWorker(job: any, workerId: string, resolveRpc: Function) {
  // 1. Give the job to the worker
  resolveRpc(job);
}
```

```

// 2. Start a countdown
const timeout = setTimeout(() => {
    this.requeueJob(job); // Worker took too long or disconnected!
}, this.props.ackTimeoutMs);

this.inProgress.set(job.id, { workerId, timeout });
}

// 3. Expose a way for the worker to say "I'm done!"
public async handleCommand(command: string, payload: any) {
    if (command === 'ACK_JOB') {
        const activeJob = this.inProgress.get(payload.jobId);
        if (activeJob) {
            clearTimeout(activeJob.timeout);
            this.inProgress.delete(payload.jobId);
        }
        return { success: true };
    }
}

```

By doing this, you have successfully built a unified, fault-tolerant Work Queue that works identically over MQTT 3.1.1 and WebSockets, entirely in user-space, keeping the developer experience perfectly aligned with your `Props` and `State` Actor model.

How should workers handle partial failures?

↩ Ja

User

What becomes the advantage of using mqtt 5 with this model?

Gemini

If you have successfully built the "RPC Pull" model, it will work perfectly as your baseline across WebSockets and MQTT 3.1.1. It gives you absolute control in user-space.

However, if you deploy Daritas into an Industrial Control System (ICS) environment that supports **MQTT 5.0**, the advantage is that you get to **delete your custom boilerplate**.

MQTT 5 was specifically redesigned to handle the exact Request/Reply (RPC) and distributed queuing patterns you are building. It moves the complexity out of your Node/WASM runtime and pushes it down to the C/Rust-optimized broker.

Here are the specific architectural advantages of MQTT 5 for your Actor model:

1. Native RPC (Correlation IDs at the Protocol Level)

In WebSockets or MQTT 3.1.1, your framework has to wrap every payload in a custom JSON envelope to track requests: `{ "correlationId": "123", "command": "PULL_JOB", "payload": {...} }` Your WASM worker has to parse that JSON, extract the ID, and route it to the Promise.

The MQTT 5 Advantage: MQTT 5 introduces native `Correlation Data` and `Response Topic` properties directly in the packet header. You send pure, raw payloads (like a binary Protobuf or flatbuffer). The broker handles the routing, and your network overhead and CPU parsing time drop significantly—which is highly valuable for constrained edge hardware.

2. Push-Based Load Balancing vs. Polling

Your fallback model is a "Pull" system. A worker asks for a job, and if the queue is empty, the connection either hangs open (consuming memory) or the worker has to poll again later (consuming network cycles).

The MQTT 5 Advantage: Using the `$share/group/topic` feature we discussed earlier, you switch to a **Push** model. The worker doesn't ask for work. It simply says, "I am alive and in the worker group." As soon as a job hits the broker, the broker pushes it to exactly one idle worker. You eliminate the latency of the "ask" and the overhead of the empty polls.

3. Native Message Expiry (TTL) for Stale Tasks

In an industrial environment, if a network link drops for 10 minutes, your `WorkQueueActor` might buffer thousands of tasks. When the link restores, the workers might waste CPU processing 10-minute-old tasks that are no longer relevant.

The MQTT 5 Advantage: You can set a `Message Expiry Interval` on the packet. If you say a task is valid for 5 seconds, and a worker isn't available in that time, the broker quietly deletes the message. You get native "stale task" prevention without writing a single `setTimeout` or timestamp-check in your TS/WASM code.

4. Negative Acknowledgments (Reason Codes)

In your fallback model, if a worker fails to process a task, it either lets the timeout expire (slow) or sends a custom RPC error back to the Queue Actor.

The MQTT 5 Advantage: MQTT 5 introduces robust Reason Codes for all operations. If a worker gets a Poison Pill payload, it can send a formal `NACK` (Negative Acknowledgment) back to the broker with a specific hex code. The broker knows exactly how to handle it based on protocol rules (e.g., routing it to a dead-letter queue) instantly, rather than waiting for an application-layer timeout.

The Strategy for Daritas

Because you are building a B2B SaaS platform that might be deployed in varying client environments, your current approach is the correct one:

1. **The Fallback (What you have):** Keep your RPC Pull model over WS/MQTT 3.1.1 as the lowest common denominator. It guarantees Daritas will run on legacy ICS networks.
2. **The Upgrade Path:** Abstract the transport interface. When the Source Edge gateway boots up, it checks the broker version. If it detects MQTT 5, it bypasses the `WorkQueueActor` polling logic and simply binds the WASM workers directly to a `$share` subscription.

You get the bulletproof reliability of the fallback, but the high-throughput, low-latency performance of MQTT 5 when the environment allows it.

How to abstract the transport layer?

↩ Ja

User

Maybe it's better to have a dedicated `WorkQueuePush` component that requires mqtt 5?

Gemini

This is an excellent architectural instinct, and in the context of Industrial Control Systems, it is almost certainly the right choice.

By creating a dedicated `WorkQueuePush` component that strictly requires MQTT 5, you avoid one of the most dangerous traps in framework design: **The Leaky Abstraction**.

If you tried to hide both the Pull (WebSocket) and Push (MQTT 5) mechanisms behind a single `WorkQueue` interface, the WASM developer wouldn't know the performance characteristics of their code. A task that takes 2 milliseconds to route on an MQTT 5 factory floor might take 50 milliseconds on a fallback WebSocket connection due to polling latency. In industrial automation, that delta can cause physical faults.

Making the component explicit gives you transparency, reliability, and clear failure boundaries.

The Explicit Component Strategy

Instead of one generic queue, you provide two distinct components in the Source Edge library. They share the same interface for the WASM worker, but their internal physics are completely different.

1. `WorkQueuePull` (The Universal Fallback)

- **Requirement:** Any transport (WebSocket, HTTP, MQTT 3.1.1).
- **How it works:** Contains all the stateful logic we discussed. It holds the `pendingTasks` array in memory, manages `ackTimeoutMs` timers, and waits for RPC `PULL_JOB` commands.
- **Trade-off:** Heavier on Edge memory and CPU. Susceptible to polling latency.

2. `WorkQueuePush` (The MQTT 5 Native)

- **Requirement:** Strict MQTT 5.0 transport.
- **How it works:** It is completely stateless. It doesn't hold arrays or run timeouts. It is essentially a syntactic sugar wrapper that translates your Actor's `props` directly into MQTT 5 protocol headers.
- **Trade-off:** Fails instantly if deployed on a legacy network.

How it looks in the Source Edge Framework

By splitting them, the `WorkQueuePush` component becomes beautifully simple. It doesn't need to manage state or handle RPC loops; it just orchestrates the broker.

TypeScript



```
// The MQTT 5 Native Push Queue
class WorkQueuePush extends SourceActor<QueueProps, never> {

  constructor(props: QueueProps) {
    super(props, null); // No state needed! The Broker holds the state.
```



```

        // Fails fast if the environment is wrong
        if (!this.transport.supportsMqtt5) {
            throw new Error("WorkQueuePush requires an MQTT 5 broker. Use WorkQueuePull")
        }
    }

    // Pushing a job just maps to a standard publish with native expiry
    public submitJob(jobPayload: any) {
        this.transport.publish(this.props.topicRoute, jobPayload, {
            qos: 1,
            messageExpiryInterval: this.props.ttlSeconds
        });
    }

    // Workers joining the queue just map to a Shared Subscription
    public registerWorker(workerId: string, handler: Function) {
        const sharedTopic = `${this.props.queueGroup}/${this.props.topicRoute}`
        this.transport.subscribe(sharedTopic, handler);
    }
}

```

Managing the Developer Experience

To keep the developer experience clean, the WASM developer shouldn't have to hardcode `new WorkQueuePush()` into their logic, because the WASM module shouldn't care where it is deployed.

Instead, you handle this at the **Deployment Configuration** level (in the Daritas cloud or Edge Gateway bootstrapper):

1. **The WASM Worker** just expects a generic `IWorkQueue` interface (which both components implement).
2. **The Edge Gateway**, upon booting in a specific factory, reads the environment.
3. If the factory has EMQX/Mosquitto v2, the Gateway injects the `WorkQueuePush` component into the worker. If it only has WebSockets, it injects `WorkQueuePull`.

This gives you the holy grail of industrial software: **Hardware-aware infrastructure with hardware-agnostic business logic**.

How to define the IWorkQueue interface?

User

How should we define the TypeScript interface for `IWorkQueue` so that both the Push and Pull components satisfy the exact same contract for the WASM workers?

Gemini

To unify the Push and Pull paradigms, you have to abstract away the control flow.

If you force the WASM worker to write a `while (true) { await queue.pull() }` loop, you are forcing it into the Pull paradigm. To make the interface agnostic, you must use an **Inversion of Control** pattern: the worker registers a handler, and the queue decides how to invoke it.

Here is the TypeScript interface that satisfies both worlds, followed by how each component implements it behind the scenes.

1. The `IWorkQueue` Interface

By using generics (`TTask` for the payload, `TResult` for the outcome), you give your WASM developers strict type safety without exposing the network layer.

TypeScript



```
export interface IWorkQueue<TTask, TResult = void> {  
    /**  
     * PRODUCER: Submits a job to the queue.  
     * Returns a promise that resolves when the queue accepts the task,  
     * NOT when the task is completed.  
     */  
    submitJob(task: TTask): Promise<void>;  
  
    /**  
     * CONSUMER: Binds a worker's logic to the queue.  
     * The Queue implementation decides whether this translates to  
     * an MQTT 5 Shared Subscription or a background RPC Pull loop.  
     */  
    registerWorker(  
        workerId: string,  
        handler: (task: TTask) => Promise<TResult>  
    ): Promise<void>;  
  
    /**  
     * Lifecycle management to detach the worker gracefully.  
     */  
    unregisterWorker(workerId: string): Promise<void>;  
}
```

2. The WASM Worker (The Consumer)

Because of the interface, the WASM developer's code is beautifully simple. They don't write polling loops, and they don't know about MQTT. They just provide the logic for their Actor's integrated mailbox to process.

TypeScript



```
class AIAnalysisActor extends SourceActor<any, any> {
  private workQueue: IWorkQueue<MathTask>;

  constructor(props: any, queueRegistry: IWorkQueue<MathTask>) {
    super(props, {});
    this.workQueue = queueRegistry;
  }

  public async onStart() {
    // We pass the handler. The queue implementation takes over from here.
    await this.workQueue.registerWorker(this.id, async (task) => {
      return await this.runHeavyMath(task);
    });
  }

  private async runHeavyMath(task: MathTask) {
    // ... heavy WASM computations ...
  }
}
```

3. How the Implementations Hide the Physics

Here is how your two distinct components satisfy that exact same `registerWorker` contract using vastly different physics.

Implementation A: `WorkQueuePush` (MQTT 5)

It maps the handler directly to a native broker feature.

TypeScript



```
class WorkQueuePush<T> implements IWorkQueue<T> {
  // ... constructor checks for MQTT 5 ...

  public async registerWorker(workerId: string, handler: (task: T) => Promise<any>) {
    const sharedTopic = `${this.groupName}/${this.topicRoute}`;

    // Let the MQTT 5 broker push messages directly to the handler
    this.transport.subscribe(sharedTopic, async (msg) => {
      try {
```

```

        await handler(msg.payload);
        this.transport.ack(msg); // Native MQTT 5 QoS 1 Acknowledgment
    } catch (error) {
        this.transport.nack(msg); // Native MQTT 5 Negative Acknowledgment
    }
    });
}
}
}

```

Implementation B: `WorkQueuePull` (WebSocket / legacy MQTT)

It hides the polling loop inside the queue class, keeping the WASM worker's thread completely clean.

TypeScript



```

class WorkQueuePull<T> implements IWorkQueue<T> {
    private activeWorkers: Set<string> = new Set();

    public async registerWorker(workerId: string, handler: (task: T) => Promise<any>) {
        this.activeWorkers.add(workerId);

        // Spin up an isolated background loop for this specific worker
        this.startHiddenPullLoop(workerId, handler);
    }

    private async startHiddenPullLoop(workerId: string, handler: (task: T) => Promise<any>) {
        while (this.activeWorkers.has(workerId)) {
            try {
                // 1. Long-poll the central coordinator via Source RPC
                const job = await this.rpc.dispatchCommand('CoordinatorId', 'PULL_JOB'

                // 2. Execute the WASM worker's logic
                await handler(job.payload);

                // 3. Acknowledge success to stop the coordinator's timeout
                await this.rpc.dispatchCommand('CoordinatorId', 'ACK_JOB', job.id);

            } catch (error) {
                // If the poll times out (queue is empty), just catch and loop again s
                // The WASM worker never sees this empty cycle.
            }
        }
    }
}

```

```
}  
}
```

The Daritas Advantage

By structuring `IWorkQueue` this way, you have achieved perfect **Separation of Concerns**.

Kim Nygren or any other algorithm auditor can review the WASM Actor's math logic without needing to understand network transports. Meanwhile, you can swap out the networking backend from a legacy WebSocket relay to a high-performance EMQX MQTT 5 broker on a factory floor, and you won't have to rewrite a single line of the edge worker code.

How to handle queue backpressure?

↩ Ja

User

If a factory generates 10,000 tasks a second but our WASM workers can only process 5,000, how do we implement backpressure in the `IWorkQueue` interface so the Edge nodes don't run out of memory?

Gemini

To prevent Edge nodes from buckling under a 10,000 task/second load, you must implement backpressure at two distinct boundaries:

1. **Consumer Backpressure:** Preventing the network from shoving too many tasks into the WASM worker's local RAM.
2. **Producer Backpressure:** Preventing the central queue (or broker) from running out of memory when tasks queue up faster than they can be processed.

Because we have cleanly abstracted the interface, we do not need to change the WASM worker's logic at all. We just need to define the failure contract for `submitJob` and leverage the specific physics of our two queue implementations.

1. Consumer Backpressure (Protecting the WASM Worker)

The consumer must strictly control how much work it accepts.

In `WorkQueuePull` (The `WebSocket` Fallback): Consumer backpressure is completely automatic. Because the internal loop we wrote uses `await handler(job.payload)`, the worker is blocked. It physically cannot ask the central queue for another job via `PULL_JOB` until the current one is finished. The worker's RAM never exceeds exactly one task in memory.

In `WorkQueuePush` (MQTT 5 Native): In a push model, a naive broker will try to blast all 10,000 messages into the worker's TCP buffer, causing an Out of Memory (OOM) crash. To fix this, we utilize a native MQTT 5 connection property called `Receive Maximum`.

When the `WorkQueuePush` component establishes its connection to the broker, it sets this property (e.g., to `10`). This tells the broker: "Do not send me a new message if I currently have 10 unacknowledged messages." The broker pauses the push, naturally buffering the excess tasks on the server instead of the Edge node.

2. Producer Backpressure (Protecting the Queue / Broker)

If the workers are protected, the backlog will inevitably pile up at the central queue. Eventually, the central queue (or broker) will run out of RAM. We must update the `IWorkQueue` interface to explicitly handle the moment the queue is full.

TypeScript



```
export class QueueFullError extends Error {
  constructor(message: string) {
    super(message);
    this.name = "QueueFullError";
  }
}
```

The producer calling `submitJob` must be prepared for this promise to reject:

TypeScript



```
// The Producer (e.g., a high-speed telemetry ingestor)
try {
  await this.workQueue.submitJob(telemetryBatch);
} catch (error) {
  if (error instanceof QueueFullError) {
    // The queue is saturated. The producer must decide what to do.
  }
}
```

3. Implementing the Queue Limits (Load Shedding)

When the queue hits its physical limit, you have to decide what happens to the overflow. In Industrial Control Systems (ICS), old data is often useless data.

In **WorkQueuePull** (The Central RPC Coordinator): We introduce a `maxQueueDepth` and a `dropPolicy` to the queue's `Props`.

TypeScript



```
// Inside the central WorkQueueActor
private handleIncomingJob(job: any) {
  if (this.jobBuffer.length >= this.props.maxQueueDepth) {
    if (this.props.dropPolicy === 'DROP_OLDEST') {
      // Ring Buffer behavior: Evict the stale task, append the new one
      this.jobBuffer.shift();
      this.jobBuffer.push(job);
    } else if (this.props.dropPolicy === 'REJECT_NEW') {
      // Signal the producer to halt
      throw new QueueFullError("Queue capacity reached.");
    }
  } else {
    this.jobBuffer.push(job);
  }
}
```

In **WorkQueuePush** (MQTT 5 Native): You configure this entirely on the broker side. Modern brokers like EMQX allow you to configure `max_mqueue_len` (Max Message Queue Length) per topic or group, and define a policy (`drop_old` vs `reject`). When a producer tries to publish to a saturated broker that is set to `reject` , the broker sends back an MQTT 5 standard Reason Code: `0x97` (Quota Exceeded) . Your **WorkQueuePush** implementation catches that code and throws the exact same `QueueFullError` back to the WASM producer.

The Daritas Standard

By implementing this, the architectural contract remains pristine. The WASM developer writing a data-ingestion routine simply wraps their `submitJob` call in a try/catch block. If they catch a `QueueFullError`, they know the system is saturated. They do not need to know if that error was generated by a local array length check or an MQTT 5 Quota Exceeded byte frame.

How should the Producer handle the `QueueFullError`?

↪ Ja

User

When a producer catches a `QueueFullError`, what are the best architectural patterns for handling it without permanently losing critical state or crashing the producer itself?

Gemini

In an Industrial Control System (ICS) or Edge architecture, simply dropping the data or crashing the producer is unacceptable. The producer is often the last line of defense before data is permanently lost.

When your WASM Actor catches a `QueueFullError`, it must transition from a "streaming" state into a "preservation" state. Here are the four standard architectural patterns for handling this gracefully, depending on the nature of the data.

1. Semantic Aggregation (Downsampling)

Best for: Continuous analog telemetry (e.g., temperature, pressure, vibration). If a temperature sensor is generating 100 readings per second and the queue is saturated, you don't actually need all 100 discrete ticks—you just need the mathematical representation of that time window.

- **How it works:** The producer catches the error and switches to an accumulator. It computes the `min`, `max`, and `average` of the failed payload and subsequent payloads.
- **The Recovery:** Once a retry succeeds, it submits a single "Aggregated Task" covering the last 5 seconds, rather than 500 individual tasks.
- **Advantage:** You survive the saturation without memory bloat and without losing the statistical reality of the factory floor.

2. Durable Local Spooling (The Store-and-Forward Pattern)

Best for: Critical discrete events that absolutely cannot be lost (e.g., configuration audits, security faults, batch completion records). If the network is saturated (or the broker goes completely offline), RAM will eventually fill up. You must flush to disk.

- **How it works:** The producer catches the `QueueFullError` and immediately writes the serialized task to local Edge storage (e.g., a local SQLite database, RocksDB, or the Origin Private File System (OPFS) if running in a browser-based WASM sandbox).
- **The Recovery:** A background process periodically checks the queue capacity. When space opens up, it pulls tasks from the local disk and submits them to the `IWorkQueue`.
- **Advantage:** Survives infinite broker outages as long as the Edge gateway has disk space.

3. Source Throttling (Backpressure Propagation)

Best for: Pull-based data ingestion (e.g., reading from local log files or querying a PLC via Modbus/OPC-UA). If you can't push data forward, stop pulling it from the source.

- **How it works:** The producer stops asking the local PLC for data. It sets a flag (`isSaturated = true`) and initiates an Exponential Backoff timer (e.g., 1s, 2s, 4s, 8s).
- **The Recovery:** Only after the timer expires does it attempt to submit a single "probe" task. If the probe succeeds, it resumes polling the PLC.
- **Advantage:** Zero memory overhead on your Edge node, pushing the buffering responsibility back to the hardware device generating the data.

4. Priority Triage (The Vital vs. Vain Strategy)

Best for: Mixed-workload producers (e.g., an Actor that emits both diagnostic logs and critical alarm states). When the ship is sinking, you drop the heavy cargo first to save the passengers.

- **How it works:** Every task is assigned a priority (e.g., 1: CRITICAL, 2: AUDIT, 3: TRACE). If a `QueueFullError` is caught, the producer consults a triage policy.
- **The Recovery:** All `TRACE` tasks are immediately hard-dropped. `AUDIT` tasks are routed to Local Spooling (disk). The producer continually retries the `CRITICAL` task using an aggressive backoff loop, utilizing whatever queue space opens up first.

Implementation Example: Aggregation + Backoff

Here is how a WASM telemetry producer might combine **Semantic Aggregation** and **Source Throttling** cleanly without blocking the event loop:

TypeScript



```
class TemperatureProducerActor extends SourceActor<any, any> {
  private queue: IWorkQueue<TelemetryTask>;
  private pendingBuffer: number[] = [];
  private isSaturated: boolean = false;

  // Fired every 10ms by a local sensor interface
  public onSensorTick(tempCelsius: number) {
    if (this.isSaturated) {
      // Queue is full. Accumulate locally to save memory.
      this.pendingBuffer.push(tempCelsius);
      return;
    }

    this.attemptSubmit(tempCelsius);
  }

  private async attemptSubmit(temp: number) {
    try {
      await this.queue.submitJob({ type: 'TEMP', value: temp });
    } catch (error) {
      if (error instanceof QueueFullError) {
        this.isSaturated = true;
        this.pendingBuffer.push(temp);
        this.startBackoffRecovery();
      }
    }
  }

  private async startBackoffRecovery() {
    let delayMs = 1000;
```

```

while (this.isSaturated) {
  await this.sleep(delayMs);

  // 1. Condense the buffered data (Semantic Aggregation)
  const aggregatedTask = this.condenseBuffer(this.pendingBuffer);

  try {
    // 2. Probe the queue
    await this.queue.submitJob(aggregatedTask);

    // 3. Recovery successful! Clear buffer and resume normal streaming
    this.pendingBuffer = [];
    this.isSaturated = false;

  } catch (error) {
    // 4. Still saturated. Double the wait time and hold the aggregated data
    delayMs = Math.min(delayMs * 2, 30000);
  }
}

private condenseBuffer(readings: number[]): TelemetryTask {
  const avg = readings.reduce((a, b) => a + b, 0) / readings.length;
  return { type: 'TEMP_AGGREGATED', value: avg, count: readings.length };
}
}

```

This ensures the actor never crashes, memory usage is strictly bounded (an array of numbers vs. thousands of full JSON tasks), and the overall data trend is perfectly preserved for Daritas's simulations.

Want to implement durable local spooling?

↪ Ja

User

How do we implement the Durable Local Spooling pattern in a WASM environment using the Origin Private File System (OPFS) or IndexedDB?

Gemini

For a high-throughput industrial platform like Daritas, choosing the right local storage primitive in a WASM sandbox is critical. If you are buffering potentially thousands of events per second during a network partition, the performance characteristics of IndexedDB and OPFS diverge sharply.

Here is the architectural breakdown and how to implement the optimal pattern.

OPFS vs. IndexedDB for Edge Spooling

IndexedDB is object-oriented and heavily tied to the JavaScript engine's garbage collector. It has high serialization overhead (structured cloning) and is strictly asynchronous. If you try to write 10,000 tasks per second to IndexedDB, you will likely cause immense memory pressure and GC pauses, ironically crashing the very edge node you are trying to save.

OPFS (Origin Private File System) is the clear winner for Industrial Control Systems. Crucially, when OPFS is used inside a Web Worker (where your WASM Actors should be running), it grants access to the `FileSystemSyncAccessHandle`. This provides **synchronous, byte-level read/write access** that mimics true POSIX file I/O (like C++ `fwrite`). It bypasses the JS event loop entirely, allowing your WASM modules to dump raw memory buffers straight to disk with near-native performance.

The Architectural Pattern: The Write-Ahead Log (WAL)

Do not try to build a complex database. The most robust store-and-forward pattern is an **Append-Only Write-Ahead Log (WAL)** with file rotation.

1. **Write (Fast):** Append serialized tasks (JSON-lines or Protobuf) to an active log file (`spool_active.log`).
2. **Rotate:** When the network returns, rename the file (e.g., `spool_processing.log`) and open a fresh `spool_active.log` for new incoming tasks.
3. **Flush (Recovery):** Read `spool_processing.log` sequentially, submit to the queue, and delete the file once successfully processed.

Implementation: The OPFS Spooler

Here is how you implement this pattern in TypeScript for your WASM workers. Note: The `createSyncAccessHandle` API strictly requires a Web Worker context.

TypeScript



```
export class OPFSSpooler {
  private rootDir!: FileSystemDirectoryHandle;
  private activeFile!: FileSystemFileHandle;
  private accessHandle!: FileSystemSyncAccessHandle;
  private encoder = new TextEncoder();
  private decoder = new TextDecoder();

  /**
```

```

    * Initializes the OPFS directory and opens the active spool file.
    */
    public async init() {
        this.rootDir = await navigator.storage.getDirectory();
        this.activeFile = await this.rootDir.getFileHandle('spool_active.log', { create: true });

        // This is the magic OPFS API that requires a Web Worker.
        // It locks the file for high-performance synchronous I/O.
        this.accessHandle = await this.activeFile.createSyncAccessHandle();
    }

    /**
     * Appends a task to the end of the log.
     * In a real WASM scenario, you might pass a raw Uint8Array here directly from memory.
     */
    public spoolTask(task: any): void {
        const payload = JSON.stringify(task) + '\n';
        const buffer = this.encoder.encode(payload);

        // Synchronously append to the end of the file
        const currentSize = this.accessHandle.getSize();
        this.accessHandle.write(buffer, { at: currentSize });

        // Force the OS to flush the buffer to physical disk
        this.accessHandle.flush();
    }

    /**
     * Rotates the log and attempts to flush it to the central queue.
     */
    public async flushToQueue(queue: IWorkQueue<any>): Promise<void> {
        if (this.accessHandle.getSize() === 0) return; // Nothing to flush

        // 1. Release the lock on the active file
        this.accessHandle.close();

        // 2. Rotate the file safely
        const processingName = `spool_processing_${Date.now()}.log`;
        // @ts-ignore - 'move' is standard but sometimes missing in older TS DOM lib
        await this.activeFile.move(this.rootDir, processingName);

        // 3. Re-open a fresh active file for new incoming tasks
        this.activeFile = await this.rootDir.getFileHandle('spool_active.log', { create: true });
        this.accessHandle = await this.activeFile.createSyncAccessHandle();

        // 4. Process the rotated file
    }

```

```

    const processingFile = await this.rootDir.getFileHandle(processingName);
    await this.processRotatedFile(processingFile, queue);
  }

  private async processRotatedFile(fileHandle: FileSystemFileHandle, queue: IWorkQueue) {
    const file = await fileHandle.getFile();
    const text = await file.text();
    const lines = text.split('\n').filter(line => line.trim() !== '');

    for (const line of lines) {
      try {
        const task = JSON.parse(line);
        // Attempt to send. If the queue rejects again, this throws
        // and aborts the flush, leaving the processing file intact.
        await queue.submitJob(task);
      } catch (error) {
        if (error instanceof QueueFullError) {
          console.warn("Queue saturated during flush. Aborting recovery.");
          // In a production system, you would append the remaining lines
          // back to the active spool or keep this file for the next attempt
          return;
        }
        throw error;
      }
    }

    // 5. Success! Delete the rotated file.
    await this.rootDir.removeEntry(fileHandle.name);
  }
}

```

Integrating with the WASM Actor

With this utility, your WASM Producer's backpressure logic becomes incredibly robust. When the `IWorkQueue` throws a `QueueFullError`, the Actor simply falls back to the spooler:

TypeScript



```

// Inside the Producer Actor
private async attemptSubmit(task: TelemetryTask) {
  try {
    await this.queue.submitJob(task);
  } catch (error) {
    if (error instanceof QueueFullError) {

```

```
        // Network or Broker is saturated. Drop to disk immediately.  
        this.localSpooler.spoolTask(task);  
        this.startRecoveryLoop(); // Starts periodic checks to call flushToQueue()  
    }  
}  
}
```

Why this fits the Daritas Model

By combining OPFS with the previously defined `IWorkQueue` interface, you maintain strict component isolation. The central Daritas orchestrator never needs to know that the Source Edge node temporarily lost connection and buffered 50,000 telemetry readings to a local file. From the orchestrator's perspective, the network simply experienced a delay, and then the data arrived perfectly sequenced via the standard MQTT 5 or WebSocket transport.

Want to optimize the serialization?

↩ Ja

User

Since JSON parsing is CPU-intensive, how should we pass raw binary formats like FlatBuffers or Protobuf directly between the WASM memory and the OPFS spooler to achieve zero-copy architecture?

Gemini

To achieve a true zero-copy architecture between your WASM edge workers and the OPFS disk, you have to completely bypass JavaScript's garbage collector and serialization engine.

When you use JSON, JavaScript has to allocate memory for the string, parse it into an object, and then serialize it again to write to disk. By switching to a raw binary format—and **FlatBuffers** is the gold standard here because it requires absolutely no parsing or unpacking step, unlike Protobuf—you treat JavaScript merely as a traffic cop directing pointers.

Here is the architectural shift required to stream FlatBuffers directly from WASM memory to OPFS.

The Paradigm Shift: Length-Prefixed Framing

In the JSON model, we separated tasks with a newline (`\n`). In a raw binary stream, a newline character could naturally occur inside the data payload. Therefore, we must switch to **Length-Prefixed Framing**.

Every write to the OPFS log will now consist of:

1. A **4-byte Header** (Uint32) stating the exact size of the payload.
2. The **Raw Payload** (the view of the WASM memory).

Step 1: The WASM Memory View (Zero-Copy JS Interop)

Your Rust/C++/Zig code compiled to WASM will construct the FlatBuffer in its own linear memory. Instead of returning data, the WASM function returns two integers to the TypeScript wrapper: the memory address (`ptr`) and the size (`len`).

TypeScript



```
// Inside your TS Actor wrapper
function handleSensorTickWasm() {
  // 1. WASM computes the FlatBuffer and returns its location in memory
  const { ptr, len } = wasmExports.generate_telemetry_flatbuffer();

  // 2. Create a zero-copy VIEW over the WASM memory.
  // This does not copy the bytes; it just creates a JS reference to them.
  const wasmMemory = wasmExports.memory.buffer;
  const payloadView = new Uint8Array(wasmMemory, ptr, len);

  // 3. Spool or send the raw view
  this.attemptSubmit(payloadView);
}
```

Step 2: The Binary OPFS Spooler

We rewrite the OPFS Spooler to accept this `Uint8Array` view and write it directly to disk using length-prefixed framing.

TypeScript



```
export class BinaryOPFSSpooler {
  private rootDir!: FileSystemDirectoryHandle;
  private activeFile!: FileSystemFileHandle;
  private accessHandle!: FileSystemSyncAccessHandle;

  // Pre-allocate a 4-byte buffer for the length header to avoid GC churn
  private headerBuffer = new ArrayBuffer(4);
  private headerView = new DataView(this.headerBuffer);

  public async init() {
    this.rootDir = await navigator.storage.getDirectory();
    this.activeFile = await this.rootDir.getFileHandle('spool_binary.log', { create: true });
    this.accessHandle = await this.activeFile.createSyncAccessHandle();
  }

  /**
   * Accepts a zero-copy view of the WASM memory and streams it to disk.
   */
  public spoolBinaryTask(payloadView: Uint8Array): void {
    const currentSize = this.accessHandle.getSize();

    // 1. Write the 4-byte length header (Little Endian)
    this.headerView.setUint32(0, payloadView.byteLength, true);
    this.accessHandle.write(new Uint8Array(this.headerBuffer), { at: currentSize });

    // 2. Write the WASM memory view directly to disk
    this.accessHandle.write(payloadView, { at: currentSize + 4 });

    // Flush to physical media
    this.accessHandle.flush();
  }
}
```

Step 3: Streamed Recovery (Avoiding OOM)

In the previous JSON example, we read the entire file into memory using `file.text()`. If a Daritas edge node was offline for hours, reading a 500MB log file into RAM all at once will crash the browser/worker.

Because we are using `SyncAccessHandle`, we can sequentially read chunk by chunk, streaming the binary directly into the `IWorkQueue`.

```
public async flushToQueue(queue: IWorkQueue<Uint8Array>): Promise<void> {
    const totalSize = this.accessHandle.getSize();
    if (totalSize === 0) return;

    let offset = 0;
    const headerReadBuffer = new Uint8Array(4);

    while (offset < totalSize) {
        // 1. Read the 4-byte header
        this.accessHandle.read(headerReadBuffer, { at: offset });
        const dataView = new DataView(headerReadBuffer.buffer);
        const payloadLen = dataView.getUint32(0, true);
        offset += 4;

        // 2. Read the exact payload
        const payloadBuffer = new Uint8Array(payloadLen);
        this.accessHandle.read(payloadBuffer, { at: offset });
        offset += payloadLen;

        try {
            // 3. Submit the raw binary to the Push/Pull queue
            await queue.submitJob(payloadBuffer);
        } catch (error) {
            if (error instanceof QueueFullError) {
                // Truncate the file to remove what we successfully processed,
                // leaving the rest for the next flush attempt.
                this.truncateProcessedData(offset - 4 - payloadLen);
                return;
            }
            throw error;
        }
    }

    // Success! Truncate the file to 0 bytes for the next round.
    this.accessHandle.truncate(0);
}
```

The Ultimate Industrial Setup

With this architecture, you have achieved an incredibly resilient data pipeline perfectly suited for your ICS tracking platform:

1. **WASM** computes the logic (perhaps algorithms Kim Nygren has audited) and formats the FlatBuffer in linear memory.

2. **TypeScript** grabs the pointer and hands it to OPFS.
3. **OPFS** writes the memory physically to the SSD.
4. **IWorkQueue** takes that exact same binary buffer and hands it to the MQTT 5 client.
5. **MQTT 5** transmits the raw bytes to the Daritas cloud.

The JavaScript thread never once parses, loops over, or interrogates the payload data. It is purely an orchestrator for memory pointers, saving your edge CPU entirely for business logic and network encryption.

Want to structure the MQTT binary routing?

↪ Ja

User

What about MessagePack ?

Gemini

MessagePack is a fantastic format, and it is frequently used in Edge/ICS environments because it is essentially "Binary JSON."

If you use MessagePack, the exact same TypeScript OPFS Spooler architecture we just built **still works perfectly**. Your WASM module packs the data, hands a `Uint8Array` pointer to JavaScript, and JavaScript streams those raw bytes directly to disk and over MQTT 5 without ever parsing them.

However, architecturally, you must understand the distinction between **Zero-Copy Transport** (which MessagePack achieves here) and **Zero-Copy Serialization** (which it does not).

Here is how MessagePack compares to FlatBuffers in your Daritas pipeline, and why you might choose one over the other.

The Core Difference: Memory Allocation on Read

With MessagePack (Requires Parsing): When the destination (e.g., your Daritas cloud orchestrator, or another WASM worker) receives a MessagePack payload, it cannot read a single piece of data directly. To find the `temperature` value, the CPU must allocate memory and **unpack (deserialize)** the entire byte array into an in-memory object/struct.

- The Cost: CPU cycles and Garbage Collection/memory allocation at the destination.

With FlatBuffers (True Zero-Copy): FlatBuffers are designed around strict offset pointers. If the cloud receives a 5KB FlatBuffer and only needs to read the `sensor_id` to route it, it does not deserialize the payload. It just reads the bytes at a specific memory offset.

- The Cost: FlatBuffers require you to pre-compile rigid schemas (`.fbs` files) into Rust/C++ code ahead of time.

Why You Might Choose MessagePack Anyway

Despite the CPU unpacking cost at the destination, MessagePack is often the right choice for an ICS platform like Daritas for three reasons:

1. **Schema-less Flexibility:** Industrial environments are notoriously messy. A PLC might suddenly start emitting a new data field, or a custom Modbus integration might have a dynamic payload structure. FlatBuffers will crash or ignore data if the strict schema doesn't match. MessagePack handles dynamic, evolving payloads effortlessly, just like JSON.
2. **Developer Experience (DX):** Maintaining FlatBuffer schema files across a distributed microservice architecture (WASM Edge, Node Gateway, Supabase/PostgreSQL backend) adds significant friction. MessagePack allows you to just pack a standard object and go.
3. **Payload Size:** For small, simple tasks, MessagePack often yields a smaller byte payload than FlatBuffers, because FlatBuffers carries the overhead of its internal vtables (pointer tables).

How it impacts the Edge Architecture

If you switch your WASM worker from FlatBuffers to MessagePack, the JavaScript orchestration layer **does not change at all**.

The `BinaryOPFSSpooler` we wrote doesn't care what the bytes mean. It just sees: `[4-byte Length Header] + [N-bytes of MessagePack Data]`

JavaScript remains completely blind to the payload, bypassing the V8 engine's garbage collector. The only difference is that your WASM developer uses a library like `rmp-serde` (in Rust) to generate the bytes, rather than

the FlatBuffers compiler.

The Verdict for Daritas

- **Choose FlatBuffers if:** You are streaming 10,000+ rigid telemetry points per second (like vibration waveforms) where the schema never changes, and CPU efficiency at the cloud ingestion layer is your ultimate bottleneck.
- **Choose MessagePack if:** Your `IWorkQueue` is handling variable workloads (e.g., a mix of telemetry, configuration audits, and alarm states), and you want the flexibility of JSON without the massive string-parsing overhead.

How to route blind binary payloads?

↪ Ja

User

What about using MessagePack and keeping the routing key outside the binary?

Gemini

This is exactly the right architectural move. This is known as the **Envelope Pattern** (or Header-Based Routing), and it is the industry standard for high-throughput brokers like Kafka, RabbitMQ, and MQTT.

By extracting the routing key (and potentially other metadata) outside the binary payload, you achieve the best of both worlds: you get the dynamic flexibility of MessagePack for the business data, while maintaining a perfectly blind, high-speed routing layer in JavaScript and the MQTT broker.

Here is how you adapt the architecture to support this seamlessly across WASM, OPFS, and MQTT 5.

1. The Envelope Contract

We update the `IWorkQueue` interface so that the WASM worker explicitly separates where the data goes from what the data is.

TypeScript



```
export interface EnvelopedTask {
  routingKey: string; // e.g., "telemetry/robot-arm-1"
  payload: Uint8Array; // The opaque MessagePack binary
  headers?: Record<string, string>; // Optional metadata (e.g., tenantId)
}

export interface IWorkQueue {
  submitJob(task: EnvelopedTask): Promise<void>;
  // ... registerWorker remains the same
}
```

When your WASM Actor is ready to send data, it passes the routing string and the MessagePack pointer to the JavaScript wrapper. JavaScript decodes the small routing string, but leaves the MessagePack binary completely untouched.

2. Perfect Mapping to MQTT 5 (WorkQueuePush)

MQTT is intrinsically designed for the Envelope Pattern. The protocol completely separates the Topic (routing key) from the Payload.

Furthermore, MQTT 5 introduced **User Properties**, which allow you to attach arbitrary key-value headers (like HTTP headers) directly to the protocol frame without polluting your MessagePack payload.

TypeScript



```
class WorkQueuePush implements IWorkQueue {
  public async submitJob(task: EnvelopedTask): Promise<void> {
    // The JS transport remains entirely blind to the MessagePack data
  }
}
```

```

        this.transport.publish(task.routingKey, task.payload, {
            qos: 1,
            messageExpiryInterval: 3600,
            userProperties: task.headers // Native MQTT 5 headers!
        });
    }
}

```

Advantage: The EMQX broker can now route, filter, and load-balance messages across your Daritas cloud architecture solely using the `routingKey` and `headers`, never wasting a single CPU cycle unpacking the `MessagePack` binary.

3. Upgrading the OPFS Spooler Framing

Because we are separating the routing key from the payload, our local disk spooler must also store the envelope. We update the **Length-Prefixed Framing** on disk to accommodate both.

A highly efficient binary framing format for OPFS looks like this: `[2-byte Topic Length] + [Topic String] + [4-byte Payload Length] + [MessagePack Bytes]`

TypeScript



```

export class EnvelopeOPFSSpooler {
    // ... setup accessHandle as before ...

    public spoolTask(task: EnvelopedTask): void {
        const currentSize = this.accessHandle.getSize();

        // 1. Encode the routing key (Topic)
        const topicBytes = new TextEncoder().encode(task.routingKey);

        // 2. Prepare Headers (Sizes)
        const headerBuffer = new ArrayBuffer(6); // 2 bytes for Topic, 4 bytes for Payload
        const headerView = new DataView(headerBuffer);
        headerView.setUint16(0, topicBytes.byteLength, true);
        headerView.setUint32(2, task.payload.byteLength, true);

        // 3. Write sequentially to OPFS disk
        this.accessHandle.write(new Uint8Array(headerBuffer), { at: currentSize });
        this.accessHandle.write(topicBytes, { at: currentSize + 6 });
        this.accessHandle.write(task.payload, { at: currentSize + 6 + topicBytes.byteLength });

        this.accessHandle.flush();
    }
}

```

When the network recovers, the recovery loop reads the 2-byte length, pulls the topic string, reads the 4-byte length, pulls the opaque MessagePack array, reassembles the `EnvelopedTask`, and hands it back to the `IWorkQueue`.

4. Handling the Legacy Fallback (WorkQueuePull)

If this node is deployed in a legacy environment using the WebSocket RPC fallback, the Envelope Pattern still applies beautifully.

Modern WebSockets support sending raw binary frames (`ArrayBuffer`). Your central RPC coordinator simply takes the `routingKey` and maps it to the correct internal Actor mailbox, passing the opaque `Uint8Array` directly into the receiving Actor's event loop.

The Bottom Line for Daritas

By keeping the routing key outside the MessagePack binary, you establish a **Zero-Copy Routing Tier**.

1. **WASM** handles the complex packing.
2. **JavaScript** handles the network I/O and OPFS disk I/O, looking only at the routing string.
3. **The Broker** routes the traffic based on the topic string.
4. **The Destination** (another WASM worker or your Supabase backend) is the only entity that ever allocates memory to unpack the MessagePack binary.

This is an extremely mature design, ensuring your edge nodes won't buckle under heavy industrial workloads.

How to handle versioning in this architecture?

↪ Ja

User

For some edge nodes TypeScript is perfect for flexibility and extreme performance is not required. But a broker node whose sole purpose is to route messages, can I do a dedicated Go or Rust implementation of my RPC?

Gemini

Absolutely. In fact, this is the exact architectural superpower unlocked by the **Envelope Pattern** we just designed.

By separating the routing key (Topic) from the business data (MessagePack), your network tier becomes entirely language-agnostic. The broker node does not need to know TypeScript, it does not run WASM, and it does not even need a MessagePack library. It only needs to know how to read length-prefixed bytes from a TCP or WebSocket stream.

Using Go or Rust for this central routing node is the industry standard for high-throughput distributed systems. Here is how the two languages compare for this specific task, and how they interact with your edge nodes.

The "Opaque Router" Pattern

When a TypeScript edge node sends a task, it pushes the envelope frame over the wire. The Go or Rust broker acts as a high-speed traffic cop. It reads the 2-byte topic length, reads the topic string, and then reads the 4-byte payload length.

Here is the critical part: **the broker reads the opaque MessagePack bytes into memory, but it never deserializes them.**

Go



```
// A Go router reading the frame from a WebSocket or TCP connection
topicLenBuf := make([]byte, 2)
io.ReadFull(conn, topicLenBuf)
topicLen := binary.LittleEndian.Uint16(topicLenBuf)

topicBuf := make([]byte, topicLen)
io.ReadFull(conn, topicBuf)
routingKey := string(topicBuf) // The broker routes based on this

payloadLenBuf := make([]byte, 4)
io.ReadFull(conn, payloadLenBuf)
payloadLen := binary.LittleEndian.Uint32(payloadLenBuf)

// The broker holds the raw bytes and forwards them to the destination socket.
// Zero CPU cycles are wasted parsing the business logic inside!
opaquePayload := make([]byte, payloadLen)
io.ReadFull(conn, opaquePayload)

broker.RouteMessage(routingKey, opaquePayload)
```

Go vs. Rust for the Broker Node

Because the broker is just shuttling []byte arrays between sockets, the performance profile of both languages is exceptional, but they optimize for different things.

Go: The Velocity Choice

- **The Fit:** Unbeatable developer velocity for concurrent network I/O.
- **The Mechanics:** You spin up a lightweight `goroutine` for every connected edge node. You use native Go channels to pass the byte arrays between the incoming socket and the outgoing queue.
- **The Trade-off:** Go uses a Garbage Collector (GC). Even though you aren't parsing complex objects, the GC still has to clean up the raw byte arrays after they are routed. In modern Go, these GC pauses are usually under 1-3 milliseconds, which is perfectly fine for 99% of deployments.

Boolean & Beyond

Rust: The Deterministic Latency Choice

- **The Fit:** The absolute highest throughput with zero GC jitter.
- **The Mechanics:** Using an async runtime like `tokio` and the `bytes` crate, Rust can achieve true zero-copy routing. You can route a payload from an incoming TCP buffer to an outgoing TCP buffer using atomic reference counting (`Arc`), meaning memory is freed instantly the microsecond the message hits the wire.
- **The Trade-off:** A significantly steeper learning curve. Managing async socket lifetimes and backpressure in Rust requires strict engineering discipline.

DEV Community

The Polyglot ICS Architecture

This hybrid approach gives you the ultimate industrial architecture:

1. **The Edge (TypeScript/WASM):** Highly flexible, easy to update, isolates third-party algorithms safely, and perfectly handles local hardware interfaces and semantic aggregation.
2. **The Core (Go or Rust):** A hardened, compiled binary whose only job is to sustain millions of connections, manage the stateful queues, and route opaque bytes without ever dropping a frame.

This benchmark breakdown explores how Rust and Go handle tail latency and throughput tradeoffs when scaling backend microservices: [Rust Throughput](#), [Go Latency](#), [.NET Tails: The Real Tradeoffs](#).

Want to define the cross-language RPC schema?

↪ Ja

YouTube video views will be stored in your YouTube History, and your data will be stored and used by YouTube according to its [Terms of Service](#)



Rust Throughput, Go Latency, .NET
Tails: The Real Tradeoffs
prod.garbage destroyer · 283
visningar



Öppna i Rust Throughput, Go Latency, .NET
Tails: The Real Tradeoffs